

ECR-[Elastic Container Registry]

- We store Docker image inside this for future use.
- this is like our repository.
- It allows developers to store, share, and deploy container images, and integrates with other AWS services to create an ecosystem for managing containerized applications.

Search 'amazon ecr public gallery' on google-->

Steps -->

-Launch Linux Instance

The screenshot shows the 'Launch an instance' page in the AWS Management Console. The breadcrumb navigation at the top reads 'EC2 > Instances > Launch an instance'. The main heading is 'Launch an instance' with an 'Info' link. Below this, a paragraph explains that Amazon EC2 allows creating virtual machines (instances) on the AWS Cloud. The page is divided into several sections: 'Name and tags' with a text input field containing 'myLinux' and an 'Add additional tags' link; 'Application and OS Images (Amazon Machine Image)' with a search bar and a 'Quick Start' tab; and a 'Summary' panel on the right. The 'Summary' panel includes a 'Number of instances' input field set to '1', and links for 'Software Image (AMI)' (Amazon Linux 2023.6.2...), 'Virtual server type (instance type)' (t2.micro), 'Firewall (security group)' (New security group), and 'Storage (volumes)'. At the bottom right, there are 'Cancel' and 'Launch instance' buttons, along with a 'Preview code' link.

EC2 > Instances > Launch an instance

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

[Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

[Recents](#) [Quick Start](#)

▼ Summary

Number of instances [Info](#)

Software Image (AMI)
Amazon Linux 2023.6.2...[read more](#)
ami-0fd05997b4dff7aac

Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

Storage (volumes)

[Cancel](#) [Launch instance](#) [Preview code](#)

--> Search ECR



Containers

Amazon Elastic Container Registry

Share and deploy container software, publicly or privately

Amazon Elastic Container Registry (ECR) is a fully managed container registry that makes it easy to store, manage, share, and deploy your container images and artifacts anywhere.

Create a repository

Create

Pricing (US)

You pay only for the amount of data you store in your public or private repositories and data transferred to the Internet.

How it works

 CloudShell [Feedback](#)

© 2024, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

--> create a private Repo--> provide name--> create repo.

[Amazon ECR](#) > [Private registry](#) > [Repositories](#) > Create private repository



General settings

Repository name

Provide a concise name. Repository names support namespaces, which is recommended for grouping similar repositories.

637423422597.dkr.ecr.ap-south-1.amazonaws.com/

8 out of 256 characters maximum (2 minimum). The name must start with a letter and can only contain lowercase letters, numbers, and special characters _-./.

Image tag mutability [Info](#)

Specify the tag mutability setting to use. When tag immutability is turned on for a repository, tags are prevented from being overwritten.

☒ Mutable

Image tags can be overwritten.

☐ Immutable

Image tags are prevented from being overwritten.

Encryption settings

 The encryption settings for a repository can't be changed once the repository is created.

Encryption configuration [Info](#)

By default, repositories use the industry standard Advanced Encryption Standard (AES) encryption. You can optionally choose to use a key stored in the AWS Key Management Service (KMS) to encrypt the images in

Amazon ECR > Private registry > Repositories

Amazon Elastic Container Registry

- Private registry
 - Repositories
 - Features & Settings
- Public registry
 - Repositories
 - Settings

ECR public gallery [↗](#)

Amazon ECS [↗](#)

Amazon EKS [↗](#)

Getting started [↗](#)

Documentation [↗](#)

Successfully created myrepo11

Private repositories (1)

View push commands Delete Actions Create repository

Search by repository substring

Repository name	URI	Created at	Tag immutability	Encryption type
myrepo11	637423422597.dkr.ecr.ap-south-1.amazonaws.com/myrepo11	December 29, 2024, 17:01:44 (UTC+05.5)	Mutable	AES-256

CloudShell Feedback © 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences 85° Search ENG IN 17:01 29-12-2024

--> connect

Instance SSH client

EC2 > Instances > i-0011a848686221b16 > Connect to instance

Connect to instance [Info](#)

Connect to your instance i-0011a848686221b16 (myLinux) using any of these options

EC2 Instance Connect Session Manager **SSH client** EC2 serial console

Instance ID [↗](#) i-0011a848686221b16 (myLinux)

1. Open an SSH client.
2. Locate your private key file. The key used to launch this instance is LinuxKeyPair.pem
3. Run this command, if necessary, to ensure your key is not publicly viewable.


```
chmod 400 "LinuxKeyPair.pem"
```
4. Connect to your instance using its Public DNS:


```
ec2-65-0-19-249.ap-south-1.compute.amazonaws.com
```

Example:

```
ssh -i "LinuxKeyPair.pem" ec2-user@ec2-65-0-19-249.ap-south-1.compute.amazonaws.com
```

Note: In most cases, the guessed username is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

CloudShell Feedback © 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

sudo su

yum update -y

yum install docker -y


```
root@ip-172-31-3-247:/home/ec2-user
Verifying : pigz-2.5-1.amzn2023.0.3.x86_64 9/10
Verifying : runc-1.1.14-1.amzn2023.0.1.x86_64 10/10

Installed:
containerd-1.7.23-1.amzn2023.0.1.x86_64 docker-25.0.6-1.amzn2023.0.2.x86_64 iptables-libs-1.8.8-3.amzn2023.0.2.x86_64
iptables-nft-1.8.8-3.amzn2023.0.2.x86_64 libcgrouper-3.0-1.amzn2023.0.1.x86_64 libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64
libnftnl-1.0.1-19.amzn2023.0.2.x86_64 libnftnl-1.2.2-2.amzn2023.0.2.x86_64 pigz-2.5-1.amzn2023.0.3.x86_64
runc-1.1.14-1.amzn2023.0.1.x86_64

Complete!
[root@ip-172-31-3-247 ec2-user]# service docker start
Redirecting to /bin/systemctl start docker.service
[root@ip-172-31-3-247 ec2-user]# vi Dockerfile
[root@ip-172-31-3-247 ec2-user]# vi index.html
[root@ip-172-31-3-247 ec2-user]# cat index.html
<html>
<h1>Welcome to ECR</h1>
</html>

[root@ip-172-31-3-247 ec2-user]# cat Dockerfile
FROM ubuntu
RUN apt update -y
RUN apt-get install apache2 -y
COPY index.html /var/www/html
EXPOSE 80
CMD ["apachectl", "-D", "FOREGROUND"]

[root@ip-172-31-3-247 ec2-user]# docker build -t img1:v1 .
[+] Building 3.7s (4/8) docker:default
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 236B 0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest 2.2s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/4] FROM docker.io/library/ubuntu:latest@sha256:80dd3c3b9c6cecb9f1667e9290b3bc61b78c2678c02cbdae5f0fea92cc6734ab 1.4s
=> resolve docker.io/library/ubuntu:latest@sha256:80dd3c3b9c6cecb9f1667e9290b3bc61b78c2678c02cbdae5f0fea92cc6734ab 0.0s
=> sha256:80dd3c3b9c6cecb9f1667e9290b3bc61b78c2678c02cbdae5f0fea92cc6734ab 6.69kB / 6.69kB 0.0s
=> sha256:ee75a10070b0fcb0bead763c5118a36bc7cc30d4c1b0749c491bbb21f1bc3e7 424B / 424B 0.0s
=> sha256:b1d3d18ab8155949d794e522b30878cf9ba52d4c1b0749c491bbb21f1bc3e7 2.30kB / 2.30kB 0.0s
=> sha256:de44b265507ae44b212defcb50694d666f136b35c1090d9709068bc861bb2d64 29.75MB / 29.75MB 0.6s
=> => extracting sha256:de44b265507ae44b212defcb50694d666f136b35c1090d9709068bc861bb2d64 0.8s
=> [internal] load build context 0.0s
=> => transferring context: 139B 0.0s
```

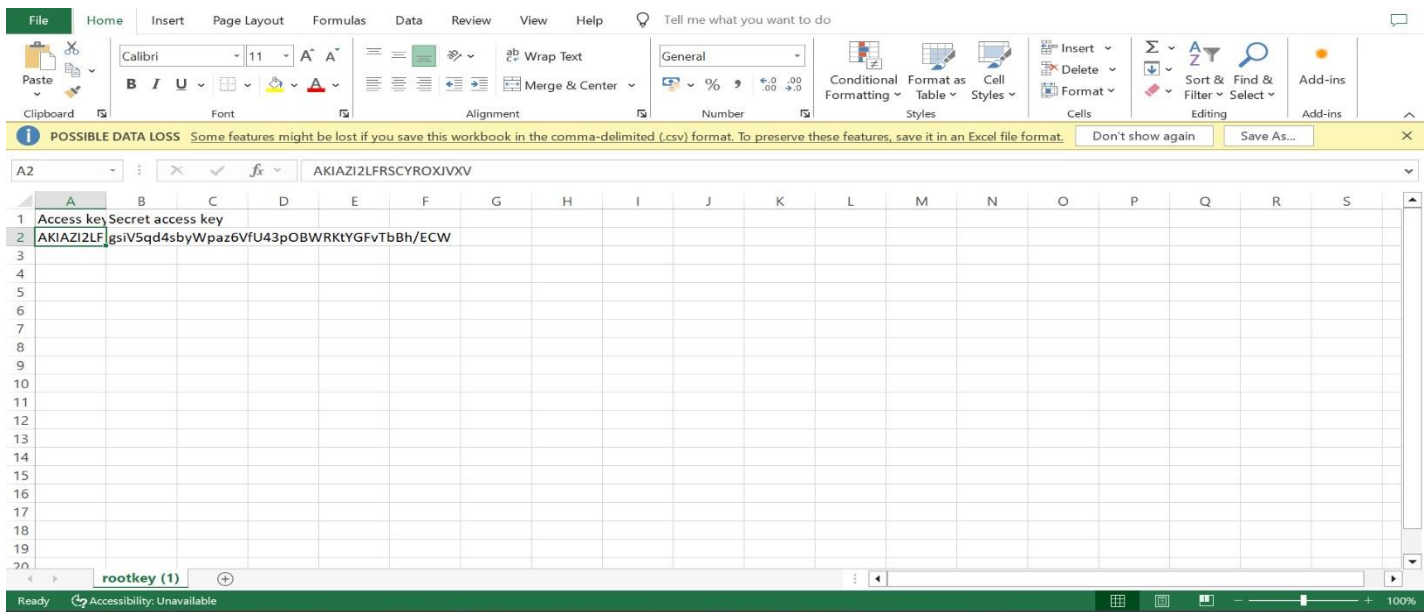
--> docker images

```
root@ip-172-31-3-247:/home/ec2-user
[root@ip-172-31-3-247 ec2-user]# service docker start
Redirecting to /bin/systemctl start docker.service
[root@ip-172-31-3-247 ec2-user]# vi Dockerfile
[root@ip-172-31-3-247 ec2-user]# vi index.html
[root@ip-172-31-3-247 ec2-user]# cat index.html
<html>
<h1>Welcome to ECR</h1>
</html>

[root@ip-172-31-3-247 ec2-user]# cat Dockerfile
FROM ubuntu
RUN apt update -y
RUN apt-get install apache2 -y
COPY index.html /var/www/html
EXPOSE 80
CMD ["apachectl", "-D", "FOREGROUND"]

[root@ip-172-31-3-247 ec2-user]# docker build -t img1:v1 .
[+] Building 26.8s (9/9) FINISHED docker:default
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 236B 0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest 2.2s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/4] FROM docker.io/library/ubuntu:latest@sha256:80dd3c3b9c6cecb9f1667e9290b3bc61b78c2678c02cbdae5f0fea92cc6734ab 2.2s
=> resolve docker.io/library/ubuntu:latest@sha256:80dd3c3b9c6cecb9f1667e9290b3bc61b78c2678c02cbdae5f0fea92cc6734ab 0.0s
=> sha256:80dd3c3b9c6cecb9f1667e9290b3bc61b78c2678c02cbdae5f0fea92cc6734ab 6.69kB / 6.69kB 0.0s
=> sha256:ee75a10070b0fcb0bead763c5118a36bc7cc30d4c1b0749c491bbb21f1bc3e7 424B / 424B 0.0s
=> sha256:b1d3d18ab8155949d794e522b30878cf9ba52d4c1b0749c491bbb21f1bc3e7 2.30kB / 2.30kB 0.0s
=> sha256:de44b265507ae44b212defcb50694d666f136b35c1090d9709068bc861bb2d64 29.75MB / 29.75MB 0.6s
=> => extracting sha256:de44b265507ae44b212defcb50694d666f136b35c1090d9709068bc861bb2d64 1.5s
=> [internal] load build context 0.0s
=> => transferring context: 139B 0.0s
=> [2/4] RUN apt update -y 6.7s
=> [3/4] RUN apt-get install apache2 -y 13.7s
=> [4/4] COPY index.html /var/www/html 0.1s
=> exporting to image 1.8s
=> => exporting layers 1.8s
=> => writing image sha256:715975a90057c8b1040f298ad37a3f06dbcf946fd26d45e1f77e47aa2a70c 0.0s
=> => naming to docker.io/library/img1:v1 0.0s
[root@ip-172-31-3-247 ec2-user]# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
img1 v1 715975a90057 13 seconds ago 229MB
[root@ip-172-31-3-247 ec2-user]#
```

--> aws configure --> paste access key --> enter --> paste secret access key --> enter --> enter --> enter-->



--> go to ecr repo--> view push commands--> copy first command and paste as it is in the putty-->

Amazon ECR > Private

Amazon Elastic Container Registry

▼ Private registry

Repositories

Summary

Images

Permissions

Lifecycle Policy

Repository tags

Features & Settings

▼ Public registry

Repositories

Settings

ECR public gallery

CloudShell Feedback

Push commands for myrepo11

macOS / Linux Windows

Make sure that you have the latest version of the AWS CLI and Docker installed. For more information, see [Getting Started with Amazon ECR](#).

Use the following steps to authenticate and push an image to your repository. For additional registry authentication methods, including the Amazon ECR credential helper, see [Registry Authentication](#).

- Retrieve an authentication token and authenticate your Docker client to your registry. Use the AWS CLI:

```
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 637423422597.dkr.ecr.ap-south-1.amazonaws.com
```

Receive an error using the AWS CLI, make sure that you have the latest version of the AWS CLI and Docker installed.
- Push your Docker image using the following command. For information on building a Docker file from scratch see the instructions [here](#). You can skip this step if your image is already built:

```
docker build -t myrepo11 .
```
- After the build completes, tag your image so you can push the image to this repository:

Copied X

Close

Image tag.

View push commands

< 1 >

Image URI Digest

Privacy Terms Cookie preferences

--> skip second command--> edit third and fourth command and paste it in putty (Edit image name and tag name only)

```
root@ip-172-31-3-247:/home/ec2-user
--> [internal] load metadata for docker.io/library/ubuntu:latest
--> [internal] load .dockerignore
--> transferring context: 2B
--> [1/4] FROM docker.io/library/ubuntu:latest@sha256:80dd3c3b9c60eb9f1667e9290b3bc61b78c2678c02cbdae5f0fea92cc6734ab
--> resolve docker.io/library/ubuntu:latest@sha256:80dd3c3b9c60eb9f1667e9290b3bc61b78c2678c02cbdae5f0fea92cc6734ab
--> sha256:80dd3c3b9c60eb9f1667e9290b3bc61b78c2678c02cbdae5f0fea92cc6734ab 6.69kB / 6.69kB
--> sha256:6e75a10070b0fcb0bead763c5118a36bc7cc10dfc1b0748c491bbb21f15c3c7 424B / 424B
--> sha256:b129df9ab81559494794e522b380878cf9ba82d4c1fb7293bcf931c3aa69ae4 2.30kB / 2.30kB
--> sha256:de44b265507ae44b212defcb50694d666f13cb35c1090d9709068bc61bb3d64 29.75MB / 29.75MB
--> extracting sha256:de44b265507ae44b212defcb50694d666f13cb35c1090d9709068bc61bb3d64
--> [internal] load build context
--> transferring context: 139B
--> [2/4] RUN apt update -y
--> [3/4] RUN apt-get install apache2 -y
--> [4/4] COPY index.html /var/www/html
--> exporting to image
--> exporting layers
--> writing image sha256:715975a90057c2bb1040f208ad37a3f06dcbfa946fd260dd5e3f77e47aa2a70c
--> naming to docker.io/library/img1:v1
[root@ip-172-31-3-247 ec2-user]# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
img1 v1 715975a90057 13 seconds ago 229MB
[root@ip-172-31-3-247 ec2-user]# aws configure
AWS Access Key ID [None]: AKIAIZ1LFRSCYR0XJVVXV
AWS Secret Access Key [None]: gsiV5qd4sbyWpaz6VfU43pOBWRKtYGFvTbBh/ECW
Default region name [None]:
Default output format [None]:
[root@ip-172-31-3-247 ec2-user]#
[root@ip-172-31-3-247 ec2-user]# aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 637423422597.dkr.ecr.ap-south-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-3-247 ec2-user]# docker tag img1:v1 637423422597.dkr.ecr.ap-south-1.amazonaws.com/myrepo11:v1
[root@ip-172-31-3-247 ec2-user]# docker push 637423422597.dkr.ecr.ap-south-1.amazonaws.com/myrepo11:v1
The push refers to repository [637423422597.dkr.ecr.ap-south-1.amazonaws.com/myrepo11]
681e1057e708: Pushed
aacc5a27c1c: Pushed
8f391455f10a: Pushed
687d50f2f6a6: Pushed
v1: digest: sha256:0ba05f82bc0ba8a5c30c541474bbce2ea588b66c5afalacbb3e407636781c9c1 size: 1160
[root@ip-172-31-3-247 ec2-user]#
```

Img:

Amazon ECR > Private registry > Repositories > myrepo11

Amazon Elastic Container Registry

Private registry

Repositories

Summary

Images

Permissions

Lifecycle Policy

Repository tags

Features & Settings

Public registry

Repositories

Settings

ECR public gallery

Image scan overview, status, and full vulnerabilities has moved to the Image detail page. To access, click an image tag.

Images (1)

Search artifacts

Image tag

Artifact type

Pushed at

Size (MB)

Image URI

Digest

☐

v1

Image

December 29, 2024, 17:15:31 (UTC+05:5)

97.29

☐ Copy URI

☐ sha256:0ba05f8...

CloudShell Feedback

© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences